

VisionControl ❖❖❖ Product Requirements Document

Contents

Product Requirements Document	1
VisionControl — AI Gesture-Based Computer Controller	1
1. Summary	1
2. Problem Statement	2
3. Goals	2
4. Non-Goals (v1)	2
5. Target Users	2
6. User Stories	3
7. Functional Requirements	3
8. Non-Functional Requirements	4
9. Technical Architecture	4
10. Success Metrics	5
11. Risks & Mitigations	5
12. Milestones / Roadmap	6
13. Open Questions	6
14. Out of Scope Confirmation	6

Product Requirements Document

VisionControl — AI Gesture-Based Computer Controller

Status	Draft v1.0
Owner	You
Last updated	2026-09-27
Category	Desktop utility / Computer Vision application

1. Summary

VisionControl lets a user control their computer’s mouse — movement, left click, and right click — using only hand gestures captured by a standard webcam. It removes the physical mouse from the interaction loop by combining real-time computer vision (OpenCV), hand landmark tracking (MediaPipe), and OS-level input simulation (PyAutoGUI).

Hand → Camera → OpenCV → Gesture → Action

2. Problem Statement

Traditional pointing devices require physical contact and free hands. There are recurring situations where that's a limitation:

- Presenting or demoing on a shared screen without wanting to hover near the laptop
- Users with limited hand mobility who can perform broad gestures more easily than fine mouse clicks
- Hands-free contexts (cooking, workshop, sterile environments) where touching a mouse/keyboard is inconvenient or unhygienic
- Hobbyist/learning interest in building a real, working computer-vision project rather than a toy demo

There is no lightweight, local, open-source tool that turns an ordinary webcam into a usable pointing device out of the box.

3. Goals

Goal	Description
G1	Track a single hand in real time from a standard laptop webcam at usable frame rates (≥ 15 FPS on mid-range hardware)
G2	Map a defined gesture set to core mouse actions: move, left-click, right-click
G3	Provide a safety mechanism to instantly pause/disable control without closing the app
G4	Ship as a runnable open-source project with clear setup instructions for non-expert users
G5	Keep the gesture vocabulary small and physically distinct enough to avoid misclassification

4. Non-Goals (v1)

- Full keyboard replacement or text input via gestures
- Multi-hand / two-handed gesture combos
- Gesture customization UI (remapping gestures to actions) — config is code-based in v1
- Cross-platform packaged installer (.exe/.dmg) — v1 ships as a Python source project
- Cloud sync, accounts, or telemetry of any kind
- Mobile/tablet support

5. Target Users

Persona	Need
Hobbyist developer	Wants a working, well-structured OpenCV/MediaPipe project to learn from and extend
Presenter/streamer	Wants to gesture-control a cursor while standing away from the laptop

Persona	Need
Accessibility-motivated user	Wants an alternative input method when fine motor control with a physical mouse is difficult

6. User Stories

1. As a user, I want to move my index finger in front of my webcam and see the system cursor follow it, so I can point at things without touching my mouse.
2. As a user, I want to pinch my thumb and index finger together to left-click, so I can select or activate UI elements.
3. As a user, I want to raise my index and middle fingers together to right-click, so I can open context menus.
4. As a user, I want to make a fist to instantly disable cursor control, so an accidental gesture doesn't move my mouse into something important.
5. As a user, I want an open palm to re-enable control, so I can resume quickly without restarting the app.
6. As a developer, I want gesture detection, mouse control, and configuration cleanly separated in code, so I can extend the gesture set or swap the output layer (e.g., game controller instead of mouse) without rewriting the vision pipeline.

7. Functional Requirements

7.1 Gesture vocabulary (v1)

Gesture	Hand shape	Action	Priority
Move	Index finger extended, others closed	Move system cursor	Must-have
Left click	Thumb tip and index tip pinched together	Single left click	Must-have
Right click	Index + middle fingers extended, others closed	Single right click	Must-have
Pause/disable	Fist (all fingers closed)	Disable cursor control	Must-have
Resume	Open palm (all fingers extended)	Re-enable cursor control	Must-have
Confirm (extensible hook)	Thumb up only	Reserved for a custom action	Nice-to-have

7.2 Core behaviors

- **FR-1:** The system must capture live webcam video and process it frame-by-frame.
- **FR-2:** The system must detect at most one hand at a time in v1 (multi-hand is out of scope).
- **FR-3:** The system must classify the current hand pose into one of the defined gestures every frame, or “no hand”/“unknown” if it doesn't match.
- **FR-4:** Cursor movement must be smoothed to reduce jitter from frame-to-frame landmark noise.

- **FR-5:** A click gesture must be debounced (cooldown period) to prevent a single pinch from firing multiple clicks.
- **FR-6:** The mapped “active zone” of the webcam frame (not the full frame) should drive cursor position, so the user doesn’t need to reach into frame corners.
- **FR-7:** The application must display a live debug view showing the camera feed, detected landmarks, current gesture, and current status, so the user can see what the system thinks it’s seeing.
- **FR-8:** The application must expose all tunable thresholds (smoothing, pinch distance, cooldown, active-zone margins) in a single configuration location, not scattered through the codebase.
- **FR-9:** The application must exit cleanly on a key press (Q) and release the camera and any OS hooks.

7.3 Setup & environment

- **FR-10:** The project must include a dependency manifest (`requirements.txt`) pinned to known-compatible versions of OpenCV, MediaPipe, PyAutoGUI, and NumPy.
- **FR-11:** The project must include one-command setup scripts for Windows and macOS/Linux that create an isolated environment and install dependencies.
- **FR-12:** Setup must detect and fail fast, with a clear message, on a known-broken environment condition (see Risks — non-ASCII path issue) rather than surfacing a raw library stack trace.

8. Non-Functional Requirements

Category	Requirement
Performance	≥ 15 FPS on a mid-range laptop CPU (no GPU required) at 960×540 capture resolution
Latency	Perceived cursor lag should stay low enough for basic pointing/clicking tasks; smoothing is tunable to trade lag vs. jitter
Reliability	A momentary loss of hand tracking (hand leaves frame) must not crash the app or leave the cursor in a runaway state
Safety	PyAutoGUI’s corner failsafe must remain enabled so a user can always abort automated cursor movement by moving the mouse to a screen corner
Portability	Must run on Windows, macOS, and Linux with Python 3.9–3.12
Privacy	All processing is local; no video, images, or gesture data leaves the machine or is persisted to disk by default
Maintainability	Vision (tracking), decision (gesture classification), and effect (OS control) logic must live in separate, independently testable modules

9. Technical Architecture

Python (runtime)

- └─ NumPy (array/matrix ops)
 - └─ OpenCV (capture + frame processing)
 - └─ MediaPipe Hands (landmark detection)
 - └─ Gesture Recognizer (landmark → gesture classification)
 - └─ Controller (gesture → PyAutoGUI mouse action)

Module responsibilities

- `hand_tracker.py` — owns the MediaPipe Hands solution; converts raw model output into pixel-space landmark coordinates.
- `gesture_recognizer.py` — pure function(s) that turn a landmark set into a named gesture plus supporting geometry (pinch distance, fingertip position). No I/O, no side effects — easy to unit test.
- `controller.py` — owns all OS-level side effects (mouse move/click), screen-coordinate mapping, smoothing, and click debouncing.
- `config.py` — single source of truth for tunable constants.
- `main.py` — orchestrates the per-frame loop and renders the debug UI.

This separation is intentional: the gesture vocabulary or the output target (e.g., swapping PyAutoGUI for a game-controller API) can change without touching the vision pipeline.

10. Success Metrics

Since this is a personal/portfolio project rather than a shipped product with analytics, success is measured qualitatively and via manual testing:

Metric	Target
Gesture recognition accuracy (manual test set of ~50 attempts per gesture)	$\geq 90\%$ correct classification
False-click rate (unintended clicks per 5-minute session of normal hand movement)	≤ 2
Time from cold start to usable cursor control	≤ 5 seconds after webcam opens
Setup success rate for a new user following the README	Works on first attempt without manual debugging

11. Risks & Mitigations

Risk	Impact	Mitigation
MediaPipe fails to load model files when the project path contains non-ASCII characters (observed: <code>em dash</code> in folder name)	Blocks app from starting; confusing stack trace	Path safety check added to <code>main.py</code> and both setup scripts; fails fast with a clear instruction to rename the folder
Poor lighting / cluttered background reduces detection accuracy	Degraded gesture recognition	Document recommended lighting in README; keep detection confidence thresholds tunable
Gesture false positives (e.g., natural hand movement misread as a click)	Accidental clicks in the wrong place	Click cooldown + fist-to-disable safety gesture; tunable pinch threshold

Risk	Impact	Mitigation
Cursor “runs away” due to a tracking glitch	User loses control of mouse mid-task	PyAutoGUI corner failsafe stays enabled; fist gesture instantly disables control
Cross-platform permission issues (macOS Accessibility/Screen Recording) block PyAutoGUI	App appears to run but does nothing	Documented in README troubleshooting section
MediaPipe/OpenCV version drift breaking compatibility	Setup fails for new users over time	Pin minimum versions in <code>requirements.txt</code> ; revisit periodically

12. Milestones / Roadmap

Phase	Scope
M1 — Core loop (done)	Webcam capture, MediaPipe hand tracking, debug overlay
M2 — Gesture set (done)	Move / pinch-click / right-click / fist / open palm classification
M3 — OS control (done)	Cursor mapping, smoothing, click debounce via PyAutoGUI
M4 — Robust setup (done)	One-click setup scripts, path safety checks, README
M5 — Extensions (future)	Air Drawing mode, Gesture Volume Control, Gesture Calculator — reusing the same tracker/recognizer architecture
M6 — Packaging (future)	Standalone executable (PyInstaller) so non-developers can run it without installing Python
M7 — Customization (future)	User-facing gesture remapping instead of editing <code>config.py</code> directly

13. Open Questions

1. Should scroll (two-finger drag, or a dedicated gesture) be added to the core gesture set before other extensions?
2. Is single-hand tracking sufficient long-term, or does a future version need two-hand support (e.g., zoom via pinch-with-both-hands)?
3. Should the “thumbs up” confirm hook ship with a default action (e.g., Enter key press) or stay unbound until a user wires it up?
4. Is a packaged executable (M6) worth prioritizing over gesture customization (M7) based on actual user feedback?

14. Out of Scope Confirmation

Explicitly not planned for v1: voice control, eye tracking, full keyboard emulation, mobile app, cloud features, or any form of usage telemetry.